MERN CHAT APPLICATION

Parts 1–47 • Complete Setup & Validation Guide

MongoDB Atlas • Clerk • Webhooks • ngrok • ImageKit • React/Vite • Express • Local Validation • Docker Preparation

Purpose

A clean, repository-ready record of the exact development journey completed before Dockerization. It is written as an operational guide so another developer can understand what was configured, why it was configured, how it was tested, and what comes next.

Security note: Real passwords, API keys, webhook secrets, private keys and connection strings are deliberately excluded. Use placeholders in documentation and keep real values in local environment variables or a secure secret store.

Architecture at a glance

Component	Development role	Endpoint / connection
React + Vite	Frontend UI	localhost:5173
Express + Node.js	REST API + webhook + Socket.IO	localhost:3000
MongoDB Atlas	Persistent database	Atlas SRV URI
Clerk	Authentication + user lifecycle events	Clerk dashboard
Clerk Webhook	Sync users to MongoDB	/api/webhooks/clerk
ngrok	Public tunnel for local webhook	HTTPS → localhost:3000
ImageKit	Media storage/delivery	Server-side API key

47-part index

Part 1 — Project overview

This guide documents the complete local setup and validation path followed for the MERN chat application before moving into containerization and Kubernetes. The application has a React/Vite frontend and a Node.js/Express backend, with MongoDB Atlas for persistence, Clerk for authentication, Clerk Webhooks for user synchronization, ImageKit for media handling, Socket.IO for real-time communication, and ngrok for exposing the local backend webhook endpoint during development.

- Repository used during the work: singhvipul2200/mern-chatapp-k8s-github-actions.
- Frontend development server: port 5173.
- Backend development server: port 3000.
- The local environment was validated before starting Docker/Kubernetes work.

Part 2 — Clone the repository

Start by cloning the project and entering the repository. The goal is to keep the original project structure intact while adding environment-specific configuration locally.

- Keep `.env` files local and out of Git.
- Use Git Bash from the Windows development machine.

Part 3 — Understand the repository structure

The application is split into backend and frontend directories. Treat them as two independently runnable applications even though they live in the same source repository.

- `backend/` — Node.js/Express API, MongoDB models, authentication routes, webhook handler and Socket.IO server.
- `frontend/` — React/Vite user interface.
- `Dockerfile` files — later used to build the two application images.
- `docs/` — recommended location for project documentation.

Part 4 — Create the MongoDB Atlas project

Create or use a MongoDB Atlas project for the chat application. The database used during validation was `mernchat`.

- Open the Atlas project and locate the cluster.
- Use Database → Browse Collections to inspect application data after the backend connects.

Part 5 — Create the MongoDB database user

Create a database user with credentials that the backend can use to authenticate to Atlas. The password should never be committed to GitHub.

- Use a strong password.
- Store the connection string only in the backend `.env` file.

- If a credential is ever exposed publicly, rotate it.

Part 6 — Configure MongoDB Network Access

Atlas must allow the development machine to reach the cluster. Configure Network Access appropriately for development.

- For a temporary development setup, allow the current development IP as appropriate.
- Avoid leaving broad access enabled for production unless it is intentionally secured.

Part 7 — Build the MongoDB URI

The backend uses a MongoDB SRV connection string. The database name in the URI was `mernchat`.

- Use the Atlas-generated URI rather than manually guessing the cluster hostname.
- Do not paste the real username/password into public documentation.

Part 8 — Configure the backend environment file

Create `backend/.env` with the values required by the application. The working environment included the server port, MongoDB URI, Clerk credentials, webhook signing secret, ImageKit private key and frontend URL.

- Keep this file out of Git.
- The actual secrets used during the setup are intentionally omitted from this document.

Safe backend `.env` template

```
PORT=3000
NODE_ENV=development
MONGO_URI=<MONGODB_ATLAS_CONNECTION_STRING>

CLERK_PUBLISHABLE_KEY=<CLERK_PUBLISHABLE_KEY>
CLERK_SECRET_KEY=<CLERK_SECRET_KEY>
CLERK_WEBHOOK_SIGNING_SECRET=<CLERK_WEBHOOK_SIGNING_SECRET>

IMAGEKIT_PRIVATE_KEY=<IMAGEKIT_PRIVATE_KEY>

FRONTEND_URL=http://localhost:5173
```

Part 9 — Backend `.env` structure

The following is a safe documentation template. Replace placeholders with the values from the corresponding provider dashboards.

- Use the exact variable names expected by the source code.

Part 10 — Configure Clerk

Create/use a Clerk application for authentication. The Clerk dashboard provides the publishable key and secret key used by the frontend/backend integration.

- The React frontend uses `VITE_CLERK_PUBLISHABLE_KEY`.
- The backend uses `CLERK_PUBLISHABLE_KEY` and `CLERK_SECRET_KEY` as required by the project.

Part 11 — Understand React vs Next.js Clerk keys

The Clerk dashboard can show different quick-start information depending on the selected framework. For this project the frontend is React/Vite, so the frontend key exposed to Vite is the publishable key. The secret key belongs on the backend only.

- Never place a Clerk secret key in a Vite frontend `.env` variable.
- Vite variables intended for browser exposure normally use the `VITE_` prefix.

Part 12 — Create/configure the Clerk webhook

The backend contains a webhook endpoint at `/api/webhooks/clerk`. Clerk must send events to this endpoint so the application can synchronize Clerk users with MongoDB.

- The webhook endpoint is exposed publicly during local development through ngrok.
- The webhook is configured in Clerk → Configure → Developers → Webhooks.

Part 13 — Create the webhook endpoint URL

The configured endpoint follows this pattern:

- `https://<your-ngrok-domain>/api/webhooks/clerk`
- The ngrok hostname can change between sessions on a free setup, so update Clerk when the public URL changes.

Part 14 — Generate the Clerk webhook signing secret

The webhook signing secret is generated by Clerk for the webhook endpoint. It is not a random value that should be invented manually.

- Open the webhook endpoint in Clerk.
- Use the endpoint's signing-secret option and copy the generated secret.
- Store it as `CLERK_WEBHOOK_SIGNING_SECRET` in the backend `.env`.

Part 15 — Why webhook verification matters

The webhook handler verifies the signature headers sent by Clerk before trusting the event body. This prevents an arbitrary client from pretending to be Clerk and writing unauthorized user data into MongoDB.

- A request without valid Clerk signature headers should fail verification.
- This behavior was observed during local testing and is expected.

Part 16 — Configure ImageKit

Create/use an ImageKit account for media uploads and delivery. During onboarding, an ImageKit ID and processing region were configured.

- The selected processing region was Mumbai, India.

- The application needs the ImageKit private key on the backend only.

Part 17 — Get the ImageKit private key

ImageKit exposes API keys from Developer Options → API keys. The private key is confidential and must not be exposed to the browser or committed to Git.

- Copy the private key from the ImageKit dashboard.
- Store it as `IMAGEKIT_PRIVATE_KEY` in `backend/.env`.

Part 18 — Configure the frontend environment

The React/Vite frontend requires the Clerk publishable key. Keep the frontend environment limited to values that are safe to expose to the browser.

- Example variable: `VITE_CLERK_PUBLISHABLE_KEY=<your_publishable_key>`.
- Do not place MongoDB credentials, Clerk secret keys, webhook signing secrets or ImageKit private keys here.

Frontend `.env` template

```
VITE_CLERK_PUBLISHABLE_KEY=<CLERK_PUBLISHABLE_KEY>
```

Part 19 — Install backend dependencies

From the backend directory, install Node.js dependencies before starting the server.

- Command: `npm install`.
- npm may report dependency vulnerabilities; treat those as a separate dependency-maintenance task unless they block the application.

Command

```
cd ~/Desktop/mern_chat_app/imessage/backend
npm install
```

Part 20 — Install frontend dependencies

From the frontend directory, install the React/Vite dependencies.

- Command: `npm install`.
- The frontend uses Vite and runs on port 5173 during development.

Command

```
cd ~/Desktop/mern_chat_app/imessage/frontend
npm install
```

Part 21 — Run the backend

Start the backend in development mode using the project's npm script. The validated server started on port 3000 and connected to MongoDB Atlas.

- Command: `npm run dev`.
- Expected log: server running on PORT 3000.

- Expected log: MongoDB connected.

Command

```
cd ~/Desktop/mern_chat_app/imessage/backend
npm run dev
```

Part 22 — Run the frontend

Start the React/Vite development server in a second terminal.

- Command: `npm run dev`.
- Expected URL: `http://localhost:5173/`.
- Vite should report that the development server is ready.

Command

```
cd ~/Desktop/mern_chat_app/imessage/frontend
npm run dev
```

Part 23 — Use a third terminal for checks

Keeping separate terminals makes troubleshooting easier: one for backend logs, one for frontend/Vite, and one for commands such as curl and ngrok.

- Terminal 1: backend.
- Terminal 2: frontend.
- Terminal 3: health checks, Git, Docker or ngrok.

Part 24 — Verify the backend health endpoint

The backend exposes `/health` and returns JSON indicating that the service is alive.

- Use: `curl http://localhost:3000/health`.
- Expected response: `{"ok":true}`.
- Typing `GET /health` directly into Git Bash is not valid shell syntax; use curl or a browser instead.

Health check

```
curl http://localhost:3000/health

# Expected:
{"ok":true}
```

Part 25 — Start ngrok

ngrok creates a public HTTPS URL that forwards traffic to the local backend. This is needed because Clerk's webhook service cannot call `localhost` directly.

- Command: `ngrok http 3000`.
- Confirm that the forwarding target is `http://localhost:3000`.
- Keep this terminal running while testing webhooks.

ngrok command

```
ngrok http 3000
```

Part 26 — Understand the ngrok forwarding flow

The public URL terminates at ngrok and is forwarded to the local Express server.

- Flow: Clerk → public ngrok HTTPS URL → localhost:3000 → Express webhook route.
- The ngrok web interface is available locally at the address shown by ngrok, commonly port 4040.

Part 27 — Configure Clerk with the ngrok URL

Copy the current HTTPS forwarding URL from ngrok and append the application's webhook path.

- Example pattern: `https://<ngrok-domain>/api/webhooks/clerk`.
- Do not use `localhost` in Clerk's webhook endpoint.

Webhook URL pattern

```
https://<your-ngrok-domain>/api/webhooks/clerk
```

Part 28 — Understand the raw-body requirement

Clerk webhook signature verification requires the original request body. Therefore the webhook route is registered before `express.json()` and uses `express.raw({ type: "application/json" })`.

- This ordering is important.
- Parsing the body first can prevent signature verification from working correctly.

Part 29 — Review the webhook implementation

The webhook handler reads `CLERK_WEBHOOK_SIGNING_SECRET`, reconstructs the request for Clerk's verifier, and handles user lifecycle events such as `user.created`, `user.updated` and `user.deleted`.

- The verified event data is then used to create/update/delete the corresponding MongoDB user record.
- Do not trust webhook data before verification succeeds.

Part 30 — Understand the webhook failure test

A manually generated curl request without Clerk's signature headers was rejected with `Webhook verification failed`. This is a positive security result, not an application failure.

- Clerk requires headers such as `svix-id`, `svix-timestamp` and `svix-signature` for verification.
- Do not disable verification just to make a manual curl request pass.

Part 31 — Use Clerk's Testing tab

The Clerk webhook endpoint provides a Testing tab where an example event can be sent to the configured endpoint. This is the correct way to test the signature path without manually forging Clerk headers.

- Select an event type such as `user.created`.

- Click Send Example.
- Review the result in the webhook endpoint's activity/overview area.

Part 32 — Confirm the webhook test succeeds

The webhook test was successfully delivered after the endpoint and signing secret were configured correctly.

- Keep backend and ngrok running during the test.
- A successful delivery confirms the public endpoint can reach the local server and the signature can be verified.

Part 33 — Log in through the application

Open the React application at `http://localhost:5173` and use Clerk's authentication UI. The Google/Gmail login flow was successfully completed during testing.

- Do not repeatedly create accounts just to test the same integration.
- An existing authenticated account is enough for the next database verification step.

Part 34 — Verify Clerk user synchronization

After an actual user is created/updated, the Clerk webhook handler synchronizes the user into MongoDB. The database should contain the Clerk user ID, email, name and profile information.

- Check the `users` collection in the `mernchat` database.
- The actual user was visible in Atlas during validation.

Part 35 — Inspect MongoDB Atlas collections

Use Atlas Data Explorer → `mernchat` → `users` to verify the application data.

- The `users` collection showed two documents during validation.
- One document represented the authenticated application user; the other corresponded to the test/example data.

Part 36 — Understand the user document

The user documents contained fields including `clerkId`, `email`, `fullName`, `profilePic`, `createdAt` and `updatedAt`. These fields demonstrate that the webhook-to-MongoDB synchronization is functioning.

- The exact IDs, email addresses and URLs are intentionally not reproduced in this documentation.

Part 37 — Test the chat UI

Once authentication and user synchronization work, use the application UI to select a conversation and send a message. This validates the application layer beyond authentication.

- Test with the available users already present in the database.
- Do not create unnecessary accounts if existing test users are sufficient.

Part 38 — Verify message persistence

After sending a chat message, inspect MongoDB Atlas and check the `messages` collection. A successful record confirms that the backend API and database persistence path are working.

- Verify sender/receiver references and message content according to the model.
- Also verify that the UI updates as expected.

Part 39 — Test media functionality

Because ImageKit was configured, test the application's image/media workflow if the project UI exposes it. Confirm that uploads complete and the returned media information can be used by the application.

- Keep the ImageKit private key server-side.
- If media upload fails, check backend logs and ImageKit configuration before changing application code.

Part 40 — Understand the final local architecture

At the end of local validation, the system has several connected services. This architecture is the baseline that Docker must reproduce.

- Browser → React/Vite frontend on 5173.
- Frontend → Express backend on 3000.
- Backend → MongoDB Atlas.
- Browser/authentication → Clerk.
- Clerk → ngrok → Express webhook.
- Backend → ImageKit for server-side media operations.

Part 41 — Prepare for Docker

Only after local validation is successful should the application be containerized. The project is intended to have separate frontend and backend images.

- Frontend image: React/Vite application.
- Backend image: Node.js/Express application.
- Do not copy `.env` files or secrets into Docker images.

Part 42 — Backend Dockerfile checklist

The backend Dockerfile should install dependencies, copy the application source and start the Node.js server. Runtime configuration should be injected when the container starts.

- Use `docker build` from the backend directory.
- Pass environment variables at runtime or through a secure deployment mechanism.
- Keep the image free of development secrets.

Part 43 — Frontend Dockerfile checklist

The frontend Dockerfile should build the Vite application and serve the resulting static files using the project's intended production server strategy.

- Be careful with Vite environment variables: values prefixed with `VITE_` can be embedded into browser assets.
- Do not treat frontend variables as secrets.

Part 44 — Build the two Docker images

Build separate images from the backend and frontend directories. Tag them consistently so they can later be pushed to Docker Hub or another registry.

- Example backend: `docker build -t <dockerhub-user>/imessage-backend:latest .`.
- Example frontend: `docker build -t <dockerhub-user>/imessage-frontend:latest .`.
- Verify with `docker images`.

Example Docker build commands

```
cd ~/Desktop/mern_chat_app/imessage/backend
docker build -t <dockerhub-user>/imessage-backend:latest .

cd ../frontend
docker build -t <dockerhub-user>/imessage-frontend:latest .
```

Part 45 — Container runtime configuration

The containerized application must use container-aware networking and environment configuration. `localhost` inside one container means that same container, not another application container.

- Backend-to-MongoDB Atlas can use the Atlas URI directly.
- Frontend-to-backend communication needs the correct reachable backend URL.
- Clerk webhook must continue pointing to a publicly reachable deployment URL, not a local container-only address.

Part 46 — Git repository documentation strategy

Keep operational documentation in a dedicated `docs/` folder so the repository root remains focused on source code and configuration. Documentation should contain safe examples rather than credentials.

- Recommended PDF path: `docs/MERN_CHAT_APP_SETUP_GUIDE.pdf`.
- Recommended companion Markdown path: `docs/MERN_CHAT_APP_SETUP_GUIDE.md`.
- Add a short link to the documentation from the root `README.md`.

Recommended repository layout

```
mern-chatapp-k8s-github-actions/
■■■ backend/
■■■ frontend/
■■■ docs/
■   ■■■ MERN_CHAT_APP_SETUP_GUIDE.pdf
■   ■■■ MERN_CHAT_APP_SETUP_GUIDE.md
■■■ README.md
■■■ ...
```

Part 47 — Milestone: local validation complete; move to Docker

Part 47 is the hand-off point from local development to containerization. The important services have been verified before Docker is introduced.

- MongoDB Atlas connection: verified.
- Backend health endpoint: verified.
- React/Vite frontend: verified.
- Clerk Google/Gmail authentication: verified.
- Clerk webhook endpoint and signature verification: verified.
- ngrok forwarding: verified.
- User synchronization into MongoDB: verified.
- Next phase: build and test the frontend and backend Docker images, then proceed toward registry deployment and Kubernetes/EKS.

NEXT PHASE → Build the two Docker images, run them locally, verify container networking/environment variables, push images to the registry, and then continue toward Kubernetes/EKS deployment.

Final pre-Docker checklist

- ■ MongoDB Atlas cluster reachable from the development machine.
- ■ Backend `.env` exists locally and is not committed.
- ■ Frontend `.env` contains only browser-safe Vite variables.
- ■ Backend starts on port 3000.
- ■ Frontend starts on port 5173.
- ■ `curl http://localhost:3000/health` returns `{"ok":true}`.
- ■ Clerk Google/Gmail authentication works.
- ■ Clerk webhook endpoint points to the current public ngrok URL.
- ■ Webhook signing verification succeeds.
- ■ An actual authenticated user is present in MongoDB `mernchat.users`.
- ■ Chat functionality has been locally tested.
- ■ Image/media functionality has been checked if enabled in the UI.
- ■ No real secrets are present in Git-tracked documentation.

Recommended GitHub repository location

Add this PDF inside the repository's **docs/** folder. Use the filename:

```
docs/MERN_CHAT_APP_SETUP_GUIDE.pdf
```

Recommended companion file:

```
docs/MERN_CHAT_APP_SETUP_GUIDE.md
```

Then add a small documentation link in the root `README.md`, for example: Documentation → MERN Chat App Setup Guide. The PDF should remain a safe, shareable record and must never contain live credentials.